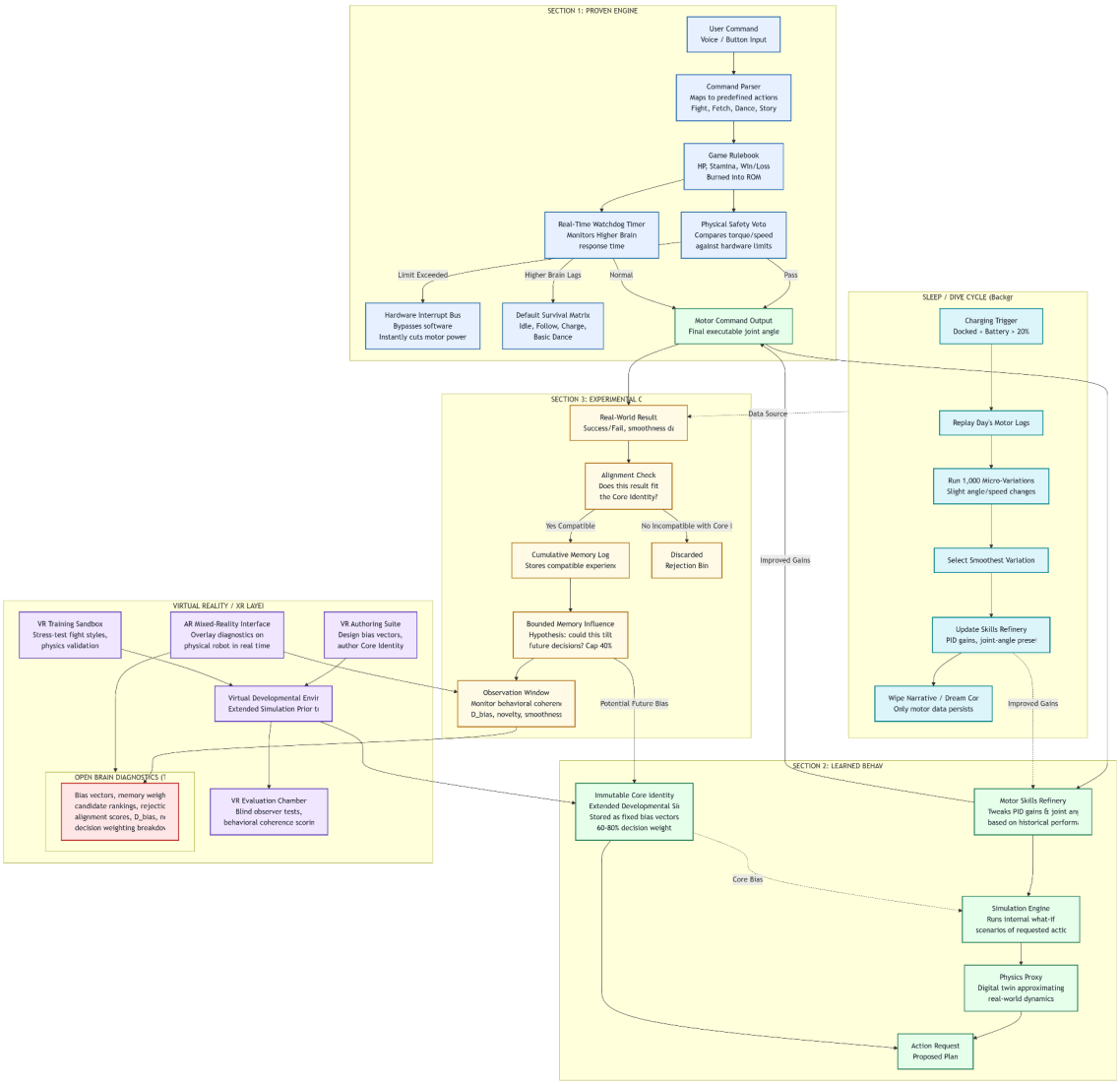


Identity-Constrained Character Robotics: An Architecture for Stable, Expressive, and Safe Consumer Humanoid Robots

Abstract

Consumer humanoid robots have remained largely stagnant in their interaction capabilities. Products such as the Ruko 6088, Tonybot, and other 17-DOF humanoid platforms offer pre-programmed movements, voice command recognition, and basic gesture control, but they lack genuine behavioral depth or the ability to learn and adapt over time. They are toys in the literal sense—programmed, predictable, and ultimately forgettable. The central challenge is that behavioral richness—movement that feels alive, decisions that feel personal—has historically required either massive generative AI models (which are unpredictable, unsafe, and expensive) or human-crafted choreography (which is limited, repetitive, and non-adaptive). Neither approach scales to a product that children can play with for years, adults can collect, and competitive players can train with in esports-style combat, without the robot breaking character or breaking physically.

This paper proposes a middle path. We present an architecture that achieves behavioral richness through a pre-authored virtual backstory, not through online learning or generative AI. This Immutable Core Identity is read-only and immutable, ensuring the character never drifts from its intended design. Physical movement is refined through offline simulation and repetition-based optimization through proven control theory (PID gain tuning over thousands of micro-simulations). The robot obeys commands like a character in a game: HP, stamina, and win/loss conditions are hardcoded in a Game Rulebook. The result is a robot that improves at physical tasks over time while remaining personality-stable, fully predictable, and physically safe.

Our work is distinct from existing approaches to anime-character robotics, such as HATSUKI, which uses imitation learning to generate actions from kinesthetic demonstrations. While HATSUKI successfully bridges the gap between figurines and interactive robots by using rear-projected anime faces and learning-based action generation, its reliance on neural networks introduces inherent unpredictability and safety concerns for consumer products. The robot can, in theory, generalize actions in unintended ways. In contrast, our architecture replaces that unpredictability with deterministic layers: a physical torque veto that compares every proposed action against absolute servo limits, a watchdog timer that defaults to safe states if the high-level brain fails to respond within 50 milliseconds, and a Game Rulebook that explicitly governs all interaction logic. The robot is an identity-constrained autonomous character system—it obeys commands, stays in character, and never invents unsafe actions.

This paper presents the architecture in full, its proposed implementation on commercially available 17-DOF platforms (e.g., Tonybot, Ruko 6088), and a comprehensive evaluation methodology. We do not claim to have achieved sentience or general intelligence. We do not claim our robot is conscious. We claim that this framework is a testable hypothesis: by storing compatible real-world experiences over time and observing whether they eventually tilt decision-making within a capped bias range, we can investigate the emergence of behavioral coherence without sacrificing predictability. This work is intended as a design and implementation paper, contributing a reproducible architecture for safe, expressive, game-governed consumer robots that appeal to children, adult collectors, and esports competitors alike. We specifically position this architecture as an alternative to the dominant paradigm of generative AI robotics, arguing that determinism, safety, and emotional resonance through authored backstory are not mutually exclusive—and that the game of interaction is the true source of value, not the illusion of general intelligence.

---

1. Introduction

1.1 The Problem of Behavioral Richness in Consumer Robotics

The consumer robotics market has produced a range of humanoid platforms designed for education, entertainment, and hobbyist use. These platforms typically offer between 15 and 20 degrees of freedom, basic sensor suites including accelerometers and ultrasonic rangefinders, and programmable interfaces that allow users to create simple movement sequences. Examples include the Tonybot, a 17-DOF humanoid with high-voltage bus servos and an open programming environment, and the Ruko 6088, a mass-market toy with wheel-based locomotion and limited articulation.

These platforms share a common limitation: their behavioral repertoire is shallow. They can execute pre-programmed movements, respond to voice commands, and perform basic gestures, but they cannot develop a consistent personality, adapt their behavior based on experience, or maintain a coherent character over extended interaction. The result is a product category that fails to sustain long-term engagement. Children lose interest after the novelty fades. Adult collectors display the robot on a shelf rather than interacting with it. Competitive players find no depth to master.

The root cause is architectural. Behavioral richness—the quality of movement and decision-making that makes a robot feel alive—has historically required one of two approaches. The first is generative artificial intelligence, particularly large language models and neural network-based controllers. These systems can produce novel, contextually appropriate behavior, but they are unpredictable, difficult to audit, and potentially unsafe. They may generate actions that exceed mechanical limits, violate social norms, or contradict established character traits. They are also computationally expensive, often requiring cloud connectivity or specialized hardware that is incompatible with consumer price points.

The second approach is human-crafted choreography. Writers and animators manually design every movement and response, creating a fixed script that the robot executes. This approach is predictable and safe, but it is also limited, repetitive, and non-adaptive. The robot cannot learn from experience, cannot develop a unique relationship with its owner, and cannot surprise or delight through emergent behavior. It is, in effect, a puppet with a fixed repertoire.

Neither approach is satisfactory. Generative AI sacrifices safety and predictability for richness. Human choreography sacrifices richness for safety and predictability. The consumer robotics market needs a third path.

1.2 A Middle Path: Authored Identity and Bounded Adaptation

This paper proposes such a path. We present an architecture that achieves behavioral richness through a pre-authored virtual backstory, not through online learning or generative AI. The robot's personality is encoded as a fixed set of bias vectors, representing a 100-year life lived in a virtual anime world before the robot was ever built. This Immutable Core Identity is stored in read-only memory and cannot be modified, patched, or updated. It is the architectural anchor that prevents personality drift.

Physical movement, in contrast, is refined through offline simulation and repetition-based optimization. During periods of inactivity—typically when the robot is placed on its charging station—the system replays the day's actions, generates thousands of micro-variations, and selects the smoothest, most efficient variations to update its motor control parameters. This process improves the robot's physical execution capabilities without altering its core personality.

The robot obeys commands like a character in a game. HP, stamina, and win/loss conditions are hardcoded in a Game Rulebook. The robot does not decide what to do

based on open-ended reasoning; it selects actions from a pre-authored repertoire, weighted by its Core Identity and its accumulated experience. The result is a robot that improves at physical tasks over time while remaining personality-stable, fully predictable, and physically safe.

1.3 Contributions

This paper makes the following contributions:

First, we present the Authored Identity Architecture, a framework for creating consumer robots with stable, expressive personalities that are authored before embodiment and remain immutable throughout the product's lifetime.

Second, we describe the Immutable Core Identity,

a read-only encoding of the robot's personality

as a set of bias vectors representing an extended

developmental simulation.

Third, we present the Simulation Engine and Physics Proxy, which generate and validate candidate actions in real time, ensuring that all movements are physically feasible and dynamically stable.

Fourth, we describe the Physical Safety Veto, Watchdog Timer, and Hardware Interrupt Bus, which provide deterministic, hardware-level safety guarantees independent of the high-level cognitive architecture.

Fifth, we present the Growth Layer, which logs real-world experiences, evaluates their compatibility with the Core Identity, and provides a mechanism for observing behavioral coherence over time without allowing unbounded personality drift.

Sixth, we describe the Sleep/Dive Cycle, which improves the robot's physical execution capabilities through offline optimization while preventing narrative persistence and identity drift.

Seventh, we provide a comprehensive implementation plan on commercially available hardware, including the Tonybot and Ruko 6088 platforms, and outline the design requirements for an ideal custom platform.

1.4 Organization

The remainder of this paper is organized as follows. Section 2 reviews related work in humanoid robotics, AI safety, and character authoring. Section 3 presents the architecture in full, including the safety layers, the Virtual Mind, the Growth Layer, and the Sleep/Dive Cycle. Section 4 describes the implementation on commercially available hardware. Section 5 discusses the theoretical foundations of Identity-Constrained Character Robotics, including the Authored Identity Architecture and the concept of behavioral coherence. Section 6 presents the evaluation methodology. Section 7 discusses limitations and future work. Section 8 concludes.

---

2. Related Work

2.1 Humanoid Robotics Platforms

Consumer humanoid robots have been available for over a decade, but their capabilities have improved slowly. Early platforms such as the RoboSapien offered basic walking and gesture control. More recent platforms such as the Tonybot and Ruko 6088 offer increased articulation, improved sensors, and open programming environments.

The Tonybot is a 17-DOF humanoid designed for education and hobbyist use. It features LX-824HV high-voltage bus servos with 17 kg.cm torque, an H5S robot controller with ESP32 expansion, and support for Arduino, Scratch, and Python programming. Its hard aluminum alloy body provides durability for repeated interactions.

The Ruko 6088 is a mass-market consumer toy with wheel-based locomotion, two motors, and an infrared sensor for obstacle detection. It supports up to 50 motion combinations and offers approximately 120 minutes of playtime. While less capable than the Tonybot, it represents the lower end of the market and demonstrates consumer expectations for movement and interactivity.

Neither platform offers behavioral richness. Both execute pre-programmed movements and respond to basic commands. Neither develops a consistent personality, adapts based on experience, or maintains a coherent character over time.

2.2 AI Safety and Alignment

The field of AI safety has developed a range of techniques for ensuring that artificial systems behave as intended. These include reward shaping, constrained optimization, interpretability tools, and formal verification. However, these techniques are primarily designed for systems that learn and adapt over time, where the goal is to prevent the system from developing harmful goals or behaviors.

Our architecture takes a different approach. Rather than attempting to align an open-ended learning system, we constrain the system's behavior from the outset. The Core Identity is immutable. The Memory Log's influence is capped. The Physics Proxy and Physical Safety Veto provide hardware-level guarantees. The result is a system that is safe by construction, not by alignment.

2.3 Character Authoring and Virtual Agents

The use of virtual environments for training and developing artificial agents is well-established. Platforms such as DeepMind Lab, AI2-THOR, HabitatSim, iGibson, and VirtualHome provide simulated worlds for training embodied AI. These environments are used for skill acquisition, reinforcement learning, and task performance optimization.

Our work repurposes virtual environments for a different function: the curated, extended incubation of a digital agent before embodiment. Rather than using simulation solely for skill acquisition, we use it as a developmental space—a womb or home in which the agent's Core Identity is formed, solidified, and strengthened over an extended period. This draws on concepts from digital twins, embodied agents in simulated environments, and physiologically grounded environments such as EVAAA.

2.4 Anime-Character Robotics

HATSUKI is a system that uses imitation learning to generate actions from kinesthetic demonstrations, combined with rear-projected anime faces. It successfully bridges the gap between figurines and interactive robots, demonstrating that anime-style characters can be embodied in physical platforms.

Our work differs in its approach to action generation. HATSUKI relies on neural networks, which introduce inherent unpredictability. Our architecture replaces that

unpredictability with deterministic layers: a pre-authored Core Identity, a Physics Proxy that validates every action against hardware limits, and a Physical Safety Veto that provides hardware-level guarantees. The result is a system that is expressive and character-consistent while remaining fully predictable and safe.

---

3. Architecture

3.1 Safety-Critical Layers

The architecture is built on a foundation of safety-critical layers that operate independently of the high-level cognitive functions. These layers are implemented in firmware on a separate microcontroller or on the ESP32's secondary core, ensuring that they remain functional even if the main processor fails.

3.1.1 Physical Safety Veto

The Physical Safety Veto is designed to prevent unsafe conditions, not to restrict dynamic movement. The distinction is critical: an action is unsafe only if it would damage the robot, injure a human, or create an unrecoverable instability. Actions that are physically demanding—flips, slides, parries, takedowns, and mounts—are permitted as long as they remain within the robot's mechanical and electrical limits.

The Veto operates on a principle of capability boundaries, not action prohibitions. It compares proposed actions against three categories of limits.

Mechanical Limits: Each servo motor has a maximum and minimum angle of rotation. If a proposed action would move a joint beyond its physical range, the action is rejected. However, within that range, the robot has full freedom. A flip requires the robot's arms to move through their full range of motion; this is permitted as long as no joint exceeds its hard stop.

Dynamic Stability Limits: The robot's center of mass must remain within its support polygon during dynamic movements. The Veto uses the Physics Proxy to evaluate whether a proposed action would cause the robot to lose balance beyond its ability to recover. However, the Veto does not reject actions that require dynamic balance adjustments so it only rejects actions where the center of mass would move outside the

recoverable envelope. A slide or a parry involves controlled instability; this is permitted as long as the robot can maintain control.

Power and Thermal Limits: Each servo has a maximum torque output and a maximum operating temperature. The Veto rejects actions that would require sustained torque above the servo's rated capacity or that would cause overheating over the expected duration of the action. However, brief high-torque bursts—such as those required for a takedown or a mount—are permitted as long as they remain within the servo's peak torque rating and are followed by adequate cooling periods.

Crucially, the Veto does not reject actions based on their danger in a game sense. A takedown is physically demanding but permitted. A mount is permitted. A flip is permitted. A slide is permitted. The robot is capable of executing the full range of dynamic movements associated with anime-style combat, constrained only by the physics of its hardware.

The Veto is also context-aware through its integration with the Physics Proxy. If the robot is on a soft surface such as carpet, the Physics Proxy provides different friction and damping parameters than if it is on a hard surface such as wood floor. The Veto uses these parameters to evaluate whether a proposed action is physically feasible in the current environment. A slide that would work on a smooth floor might be rejected on carpet; the same slide on a smooth floor is permitted.

3.1.2 Watchdog Timer

The Watchdog Timer provides a hardware-level guarantee that the robot will transition to a safe state if the high-level cognitive architecture fails to respond within a specified time window. The timer is configured with a 50-millisecond timeout. The main processor must reset the timer at least once every 50 milliseconds. If it fails to do so, the timer triggers a hardware interrupt that disables the main processor's output and transfers control to the Default Survival Matrix.

The Default Survival Matrix is a state machine implemented in read-only memory on the secondary microcontroller. It has no connection to the main processor's memory or processing units and cannot be modified by software. Its states include Idle, Follow, Charge, and Basic Dance. These states are designed to be safe under all conditions and to allow the robot to continue functioning at a basic level even if the main processor is unresponsive.

3.1.3 Hardware Interrupt Bus

The Hardware Interrupt Bus provides a direct electrical connection from the IMU, thermal sensors, and current sensors to the motor power controllers. It uses analog comparators with fixed thresholds, ensuring that safety responses occur within 1 millisecond regardless of software state. If the IMU detects an acceleration beyond a safe threshold, if a thermal sensor detects overheating, or if a current sensor detects a short circuit, the interrupt bus immediately cuts power to the affected servos.

This layer operates entirely in hardware. It does not depend on software, firmware, or the main processor. It is the last line of defense against catastrophic failure.

3.1.4 Customizable Game Settings

The Game Rulebook is not monolithic. While the core safety layers are immutable, the game parameters—HP, stamina costs, damage values, win/loss conditions, and action costs—are stored in a separate, user-accessible configuration space. This allows owners to create custom rule sets for their robots, enabling varied gameplay styles and competitive formats.

Users can connect to the robot via a computer or mobile application to adjust game parameters. The interface provides presets such as Standard, Fast Combat, Endurance, and Tournament, as well as custom sliders for HP values (50-500), stamina regeneration rates, damage values for each action, action stamina costs, and win/loss conditions.

For esports-style tournaments, the game settings can be locked to a standardized configuration. This ensures fair competition across different robot units. Tournament organizers can distribute a configuration file that all robots must load before competing.

The configuration interface enforces safety bounds on all adjustable parameters. While users can increase HP, they cannot increase the robot's torque output beyond hardware limits. Users can make the game more forgiving or more punishing, but they cannot make the robot physically unsafe. This is enforced by the Physical Safety Veto, which overrides any game settings that would conflict with hardware limits.

3.2 The Virtual Mind

The Virtual Mind is the expressive core of the architecture. It is responsible for translating the player's commands and the game state into fluid, character-consistent motion plans that reflect the robot'sExtended Developmental Simulation (EDS)

Unlike the hardcoded layers described in Section 3.1, the Virtual Mind is implemented in software on the main processor. However, its foundational identity component is immutable and read-only, ensuring that the character's core personality remains stable over the product's lifetime.

The Virtual Mind consists of three interconnected subsystems: the Immutable Core Identity, which encodes the character's personality, preferences, and knowledge; the Simulation Engine, which generates and evaluates candidate action plans; and the Physics Proxy, which grounds those plans in real-world physical constraints.

3.2.1 Immutable Core Identity

The robot's personality is encoded as a fixed set of bias vectors stored in read-only memory. These vectors represent the character's extended developmental simulation, a life lived in an anime-style simulation world before embodiment. The Core Identity is the architectural anchor that prevents personality drift: it is burned into ROM at manufacturing time and can never be updated, patched, or modified.

The extended developmental life is not stored as a narrative text or a sequence of events. It is stored as a compressed, abstracted constraint graph that captures the character's learned dispositions across a wide range of situational contexts. Each node in the graph corresponds to a specific context—defined by spatial, social, and game-state features—and contains a probability distribution over possible responses.

Consider a character whose 100-year backstory includes training as a swordsman in a fantasy anime world. The constraint graph encodes combat context: when facing an opponent at close range with a weapon, the character biases toward aggressive, high-risk actions (80% aggressive, 15% defensive, 5% flee). Social context: when interacting with a known friendly entity, the character biases toward warm, playful responses (90% warm, 5% teasing, 5% indifferent). Spatial context: when the character's backstory includes frequent duels in arena environments, the character has a learned predisposition to use the full space, preferring open areas over confined corners.

These bias vectors are not simple sliders. They are multidimensional tensors that encode nuanced relationships between context features. The exact encoding scheme—whether it is a set of conditional probability tables, a neural embedding, or a symbolic rule base—is an implementation detail that can be chosen based on hardware constraints and desired expressiveness. The critical architectural property is that the

encoding is fixed and unlearnable. The robot can add new experiences to its Cumulative Memory Log, but those experiences can only adjust decision weights within a capped range; they cannot rewrite the Core Identity.

The Core Identity also encodes semantic knowledge of the physical world. The robot knows that a wooden sword is a weapon, that it is held in the hand, that it has a specific weight, balance, and reach, and that it can be used for thrusting, slashing, and parrying. This knowledge is not learned in the real world; it is part of the character's pre-embodiment training.

The Core Identity's bias vectors encode the character's preferred fight style, which is a direct reflection of its developmental simulation training. A character who was trained as a samurai in a rigorous dojo will favor precise, efficient strikes and defensive counters. A character who was a street brawler will favor aggressive, unpredictable attacks and opportunistic grappling. A character who was a showman will favor flashy, acrobatic movements and taunts.

The fight style is not a fixed script; it is a set of probability biases that influence action selection across a wide range of contexts. When the Simulation Engine generates candidate actions, it weights them according to the Core Identity's biases. A samurai-trained character will have a higher weight for precise thrusts and lower weight for wild swings; a brawler will have the opposite distribution.

This means that even when executing the same command, different robots with different Core Identities will produce observably different behaviors—not because they are learning or adapting, but because they are biased by their pre-authored backstories. The diversity of possible characters is limited only by the number of virtual backstories that can be authored and the richness of the bias-vector encoding.

3.2.2 Simulation Engine

The Simulation Engine is the generative core of the Virtual Mind. When the Game Rulebook requests an action, the Simulation Engine generates a large set of candidate action plans—typically on the order of 1,000 candidates—each representing a different way to execute the requested action with a different stylistic emphasis, timing, and trajectory.

The Simulation Engine generates candidates by sampling from a motion primitive library—a set of pre-authored, parameterized movement templates that define the kinematic structure of common actions. Each primitive specifies a sequence of joint

angles over time but leaves certain parameters free for variation: speed, amplitude, timing, and style.

Each candidate action is assigned a weight based on its alignment with the Core Identity's bias vectors. The Simulation Engine computes a similarity score between the candidate's style parameters and the character's preferred fight style. Candidates that match the character's biases receive higher scores.

The Simulation Engine does not generate actions in a vacuum. It integrates the robot's real-time spatial awareness via a semantic spatial map constructed from camera input, infrared distance sensors, and inertial data to ensure that candidates are contextually appropriate. The spatial map provides the robot's current pose and center of mass, the positions and velocities of nearby obstacles and surfaces, the location and orientation of objects, and the positions and movement patterns of other agents.

The Simulation Engine filters out candidates that would result in immediate collision with obstacles or that would place the robot in an unrecoverable instability. The remaining candidates that are spatially feasible and personality-aligned are passed to the Physics Proxy for detailed physical validation.

The Simulation Engine also adapts its candidate generation based on the action context—the game state, the opponent's behavior, and the history of previous actions. This allows the robot to respond dynamically to changing situations without deviating from its core identity.

If the robot is losing in HP, the Simulation Engine may generate more defensive candidates or more desperate, aggressive candidates, depending on the character's fight style. If the opponent is aggressive, the Simulation Engine may generate more counter-attack candidates. If the opponent is defensive, it may generate more feints and probing attacks.

Crucially, this adaptation is deterministic and traceable. The Simulation Engine's decision-making is driven entirely by the Core Identity's fixed biases and the current game state; there is no unsupervised learning or emergent behavior that could lead to unpredictable outcomes. An external observer—or a player—can reason about why the robot chose a particular action based on the known biases and the game context.

3.2.3 Physics Proxy

The Physics Proxy is a digital twin of the robot's physical body, running a simplified but accurate physics simulation. Its purpose is to validate candidate actions before they are executed, ensuring that they are physically feasible and dynamically stable.

The Physics Proxy models the robot as a multi-body kinematic chain with joint limits, torque limits, inertia and mass, contact dynamics, and ground reaction forces. It is not a full finite-element simulation; it is a computationally efficient approximation that runs on the robot's main processor in real time. It uses a simplified dynamic model—typically a spring-damper model with rigid body dynamics—that captures the most critical physical behaviors without excessive computational cost.

Each candidate action from the Simulation Engine is passed through the Physics Proxy, which simulates the action's execution over its full duration. The Proxy evaluates feasibility, stability, collision, and efficiency. If any parameter exceeds its limit at any point during the action, the candidate is rejected.

The Physics Proxy is specifically designed to validate dynamic movements—flips, slides, parries, takedowns, and mounts—that push the robot to its physical limits. These actions are not rejected simply because they are dynamic; they are rejected only if they exceed the robot's actual physical capabilities.

For a flip, the Physics Proxy checks whether the robot's center of mass is positioned correctly to rotate around its support point, whether the servos can generate enough torque to complete the rotation within the required time, and whether the robot will land within its recoverable stability envelope.

For a takedown, the Physics Proxy checks whether the robot has enough mass and leverage to unbalance the opponent, whether the servos can generate the required force without exceeding their torque limits, and whether the robot will maintain its own balance during the takedown.

Once the Physics Proxy has validated all candidates, it assigns each a feasibility score that combines physical efficiency, stability margin, and alignment with the Core Identity's preferences. The highest-scoring candidate is selected and passed to the Physical Safety Veto for final hardware-level validation. If the Veto passes it, the action is sent to the motor controllers for execution. If the Veto rejects it, the robot falls back to the Default Survival Matrix.

3.3 The Growth Layer

The Growth Layer is the architectural component that distinguishes our system from both purely deterministic robots and open-ended learning systems. It is designed not to create sentience or general intelligence, but to observe whether behavioral coherence—a measurable, consistent bias in decision-making that reflects accumulated real-world experience—emerges over extended operation. This is the unknown quadrant of our architecture, explicitly labeled as a hypothesis rather than a guarantee.

The Growth Layer consists of four subsystems: the Real-World Result Logger, the Alignment Check, the Cumulative Memory Log with Rejection Bin, and the Observation Window.

3.3.1 Real-World Result Logging

Every action executed by the robot generates a data record that is logged in the Real-World Result subsystem. This log is the raw material for all subsequent growth-layer processing. The log entry for each action includes execution data, environmental data, and performance metrics.

Execution data includes the action command received, the selected candidate action plan, the actual executed joint angles and velocities, the action's success or failure at its objective, and the game state before and after execution.

Environmental data includes the robot's pose and center of mass during execution, the positions and velocities of nearby obstacles, surfaces, and agents, the surface type and friction coefficient at the robot's contact points, and the locations and orientations of objects within the robot's spatial map.

Performance metrics include motor smoothness metrics derived from IMU data, energy consumption, and timing accuracy.

This log is stored in non-volatile memory and persists across power cycles. The log is structured as a time-series database, indexed by timestamp and action type, to enable efficient querying and analysis.

3.3.2 Alignment Check

Periodically—and after every action, in our proposed implementation—the logged real-world result is passed through the Alignment Check. The Alignment Check evaluates whether the executed action is compatible with the robot's Core Identity. This

is not a binary yes/no decision based on a single threshold; it is a continuous similarity score.

The Alignment Check computes a similarity score between the executed action's feature vector and the Core Identity's bias vectors for the relevant context. The feature vector encodes the action's stylistic parameters, the action's outcome, and the environmental context. The similarity metric is, for example, a cosine similarity or a Mahalanobis distance between the action's feature vector and the Core Identity's expected distribution for that context. The specific metric is an implementation choice, but it must be deterministic and transparent.

A compatibility threshold determines whether an action is considered aligned with the Core Identity. Actions with a similarity score above the threshold are marked as compatible; actions below the threshold are marked as incompatible.

Crucially, the threshold does not affect the robot's behavior in the moment—actions are executed regardless of whether they will later be marked as compatible. The Alignment Check is retrospective; it does not veto actions. This ensures that the robot can explore novel behaviors without being punished for stepping slightly outside its comfort zone.

3.3.3 Cumulative Memory Log and Rejection Bin

Compatible experiences—those that pass the Alignment Check—are stored in the Cumulative Memory Log. Incompatible experiences are discarded into the Rejection Bin, where they are logged for debugging and analysis but do not influence future behavior.

The Cumulative Memory Log is a non-volatile database that grows over the robot's lifetime. Each entry contains the compatible experience's feature vector, a timestamp and context identifier, the similarity score assigned by the Alignment Check, and the game state at the time of execution.

The Rejection Bin stores incompatible experiences. These entries include the same data as the Cumulative Memory Log, but they are flagged as rejected. The Rejection Bin serves two purposes: debugging and threshold tuning. The Rejection Bin does not influence the robot's behavior.

3.3.4 Observation Window

The Observation Window is a monitoring interface—implemented as a software dashboard accessible via a computer or mobile application—that tracks the contents of

the Cumulative Memory Log over time. The Observation Window is the primary tool for investigating the central hypothesis of our architecture: as the Cumulative Memory Log accumulates sufficient compatible experiences, the robot's decision-making may begin to show measurable, consistent biases beyond the original Core Identity—biases that reflect the character's real-world history rather than just its virtual backstory.

The Observation Window displays several quantitative indicators that can be tracked over days, weeks, and years: log size, action type distribution, smoothness trend, novelty rate, and bias drift.

Log size is the total number of compatible experiences stored. Action type distribution is the proportion of compatible experiences that are aggressive, defensive, cooperative, and so on. This can be compared against the Core Identity's expected distribution. Smoothness trend is the average motor smoothness score of compatible experiences over time. If this trend is increasing, it suggests that the robot is becoming more physically refined. Novelty rate is the proportion of actions that are novel—that have low similarity to the Core Identity's biases but still pass the compatibility threshold. A high novelty rate suggests the robot is exploring new behaviors within its character's bounds. Bias drift is a statistical measure of whether the robot's action selection is drifting toward a distribution that is measurably different from the Core Identity's initial biases.

A key design principle of our architecture is that the behavioral coherence hypothesis is falsifiable. We define clear criteria for what would constitute evidence against the hypothesis.

If, after a significant number of compatible experiences, the robot's action selection distribution remains statistically indistinguishable from the Core Identity's initial biases, the hypothesis is rejected.

If the robot's novelty rate stabilizes at zero, indicating that it is generating only actions that were already well-covered by the Core Identity, the hypothesis is rejected.

If human observers, in blind tests, cannot distinguish the robot's behavior after 10,000 actions from its behavior on day one, the hypothesis is rejected.

If any of these conditions hold, the hypothesis is rejected, and we conclude that the Growth Layer does not produce behavioral coherence. This is a valid and valuable outcome: it would suggest that behavioral coherence, if it exists, requires mechanisms beyond simple experience accumulation.

3.3.5 Capped Influence

The Growth Layer is designed to be a capped influence on decision-making, not an open-ended learning system. This is the critical safeguard that prevents the robot from drifting away from its Core Identity.

Compatible experiences stored in the Cumulative Memory Log influence future decisions through a weighting mechanism that is explicitly bounded. When the Simulation Engine generates candidate actions, it weights them based on the Core Identity's bias vectors (fixed, 60-80% of total weight) and the Cumulative Memory Log's compatibility scores (variable, but capped at 20-40% of total weight).

The exact weighting ratio is a design parameter. A higher ratio for the Core Identity makes the robot more conservative and less adaptive; a lower ratio makes it more adaptive but increases the risk of drift. We propose a default ratio of 70:30, with the Memory Log's influence capped at an absolute maximum of 40%.

The Memory Log's influence is capped in two ways. First, the weight cap: the Memory Log's contribution to the action selection score cannot exceed the absolute cap. Even if the Memory Log strongly favors a particular action, the Core Identity's contribution ensures that the robot never completely abandons its character. Second, temporal decay: older experiences are given lower weight than more recent experiences. This prevents the robot from being overly influenced by a single early experience that may have been anomalous. A decay function ensures that the robot's behavior is driven primarily by recent, relevant experiences.

Crucially, the Memory Log never modifies the Core Identity itself. The Core Identity's bias vectors remain read-only and immutable. The Memory Log only influences the weighting of candidate actions during the Simulation Engine's generation phase; it does not rewrite the constraints that define the character's fundamental personality.

This design ensures that even after years of operation, the robot's Core Identity remains exactly as it was on the day it was unboxed. The robot may develop a slight statistical preference for actions that have worked well in its real-world experience, but its underlying character—its values, its fight style, its social dispositions—remains unchanged.

3.3.6 Open Brain Diagnostics

The architecture provides continuous visibility into its internal decision-making process. At any time the developer, player, or researcher can inspect current Core Identity bias vectors, current Memory Log influence weights, action candidate rankings, reasons for

candidate rejection, alignment scores, bias drift metrics, novelty metrics, memory retrieval traces, and decision weighting breakdowns.

Unlike modern large language models, whose internal reasoning is largely opaque, the proposed architecture exposes every major decision variable for inspection and auditing. This allows longitudinal observation of behavioral development and provides a transparent substrate for studying identity stability, adaptation, and behavioral drift.

3.4 The Sleep/Dive Cycle

The Sleep/Dive Cycle is the mechanism by which the robot's physical execution capabilities improve over time without altering its core personality or introducing unpredictability. During periods of inactivity—typically when the robot is placed on its charging station—the system enters a background processing state that replays the day's actions, simulates thousands of micro-variations, and selects the smoothest, most efficient variations to update its motor control parameters. Critically, all narrative and dream content generated during this cycle is stored in volatile RAM and wiped upon wake-up, ensuring that the robot's Core Identity remains pristine while its physical skills mature.

The Sleep/Dive Cycle consists of five stages: Charging Trigger and Replay, Micro-Variation Simulation, Best Variation Selection, Motor Update, and Dream Content Wiping.

3.4.1 Charging Trigger and Replay

The Sleep/Dive Cycle is triggered automatically when the robot is placed on its charging station and the battery level exceeds a minimum threshold. This ensures that the cycle does not drain the battery during active use and that the robot has sufficient power to perform the computational work required for simulation.

The cycle begins when the robot is physically docked with its charging station, the battery charge level is above 20%, and the robot has been in a resting state for at least 30 seconds.

The first stage of the cycle is to replay the day's motor logs—the chronological record of all executed actions and their real-world results. The replay is not a physical re-execution; it is a software-based reconstruction of the actions using the logged joint angles, velocities, and environmental data.

The replay serves two purposes: data compilation and baseline generation. The robot compiles a dataset of all actions executed since the last Sleep/Dive Cycle, organized by action type and context. For each action type, the robot computes a baseline performance metric—average smoothness, torque efficiency, success rate—against which the micro-variations will be compared.

3.4.2 Micro-Variation Simulation

The second stage is the core of the Sleep/Dive Cycle: running a large number of micro-variations of each replayed action. The Simulation Engine is repurposed for this offline optimization task, but with a different objective: instead of generating diverse stylistic candidates for immediate execution, it generates slight modifications to the executed actions to identify the most efficient and smooth execution parameters.

For each action type, the Simulation Engine generates a set of variations—typically 1,000 variations per action instance, though this number is adjustable based on available processing power and time. Each variation makes small, controlled adjustments to the action's parameters: joint angles, velocities, timing, amplitude, and posture with other added as needed

The variations are generated systematically, not randomly. The parameter space is sampled using a deterministic grid or a low-discrepancy sequence to ensure comprehensive coverage without relying on random number generation. This determinism is a key design choice: it ensures that the same input always produces the same set of variations, eliminating unpredictability in the optimization process.

Each variation is passed through the Physics Proxy, which simulates the action's execution and evaluates its physical feasibility and performance. The Physics Proxy checks feasibility, smoothness, torque efficiency, and dynamic stability.

Each variation is assigned a performance score that combines these metrics into a single value. The scoring function is weighted: smoothness and efficiency are prioritized, but stability and feasibility are absolute requirements.

3.4.3 Best Variation Selection

Once all variations have been evaluated, the Sleep/Dive Cycle selects the best variation for each action type—the one with the highest performance score that remains physically feasible and stable.

The selection criteria are physical feasibility, performance score, and conservatism. If the best variation differs from the original action by more than a pre-defined threshold, the robot flags it for manual review. This prevents the optimization from making drastic changes that could be unsafe or unnatural.

The selected variation is compared to the baseline performance metric from the Replay stage. If the variation's performance score is not significantly better than the baseline, the robot may skip the update to avoid unnecessary changes. The significance threshold is configurable, but we propose a default of 5% improvement in smoothness or efficiency.

3.4.4 Motor Update

The selected variation is used to update the robot's Skills Refinery—a persistent database of optimized motor plans stored in non-volatile memory.

The Skills Refinery maintains a set of motion primitives for each action type, each with associated PID gain parameters and joint-angle presets. When the robot executes an action in the future, it uses the optimized parameters from the Skills Refinery rather than the original, unoptimized parameters.

The Skills Refinery updates the PID gains for each servo joint. PID gains determine how aggressively the servo corrects errors between the commanded and actual positions. By tuning these gains based on the optimized variation, the robot can achieve smoother, more responsive movements.

The PID gain update is incremental: the new gains are a weighted average of the old gains and the gains that would be required to achieve the optimized variation. This prevents sudden, jarring changes to the robot's control behavior and ensures that the optimization is stable.

3.4.5 Dream Content Wiping

This is the final and most critical stage of the Sleep/Dive Cycle. During the simulation phase, the robot may generate dream content—fictional narratives, imagined social interactions, or hypothetical scenarios that provide context for the optimized movements. This dream content is a byproduct of the Simulation Engine's operation and, if stored, could contribute to personality drift.

If dream content were persisted across Sleep/Dive Cycles, the robot would accumulate a growing body of false memories—experiences that never actually happened. Over time, these false memories could begin to influence the robot's decision-making, pulling it away from its Core Identity and creating unpredictable behavior. This is the exact problem our architecture is designed to prevent.

To prevent narrative persistence, the Sleep/Dive Cycle's final stage is to wipe all dream content from volatile RAM. Only the optimized motor data like the new joint-angle presets and PID gains is written to non-volatile storage. The narrative context that generated those optimizations is discarded.

The wiping mechanism is implemented as a hardware-level memory reset: upon completion of the Motor Update stage, the Sleep/Dive Cycle triggers a reset of the RAM bank used by the Simulation Engine. This reset is atomic and irreversible; once executed, the dream content cannot be recovered.

The combination of Motor Update and Dream Wiping achieves the core design goal of the Sleep/Dive Cycle: the robot physically improves at tasks over time, but its identity remains exactly the same as the day it was unboxed. The robot can execute a Fight action with greater smoothness and efficiency after a hundred Sleep/Dive Cycles, but it will still execute that action with the same personality, fight style, and value system as its original extended developmental simulation self.

3.4.6 Performance and Power Considerations

The Sleep/Dive Cycle is computationally intensive. Running 1,000 variations per action type across all actions executed in a day could require significant processing power and energy. We address this with three design optimizations.

Prioritization: Not all actions are equally important to optimize. The Sleep/Dive Cycle prioritizes actions based on their frequency of use and their physical intensity. Actions that are executed frequently receive lower priority than actions that are physically demanding. This ensures that the most impactful optimizations are performed first.

Incremental Processing: The Sleep/Dive Cycle does not need to complete all optimizations in a single session. If the robot is disconnected from its charger before the cycle is complete, the current state is saved and the cycle resumes on the next charging session. This makes the system robust to interruptions.

Power Budgeting: The Sleep/Dive Cycle is designed to operate within the robot's power budget. The cycle is throttled based on battery level and charging rate. If the battery is low or charging slowly, the cycle reduces the number of variations or extends the processing duration to avoid depleting the battery. The robot always prioritizes battery safety over optimization completeness.

---

4. Implementation

4.1 Hardware Platform Selection

The architecture is designed to be platform-agnostic, but practical implementation requires a specific hardware baseline. We identify two commercially available platforms that serve as suitable starting points for prototyping, followed by a discussion of the ideal custom platform for production.

4.1.1 Tonybot: A 17-DOF Humanoid Platform

The Tonybot is a programmable humanoid robot designed for educational and hobbyist use, offering a robust combination of articulation, computational resources, and expansion capabilities. It serves as the primary prototyping platform for our architecture due to the following specifications.

Degrees of Freedom: 17 DOF, providing sufficient articulation for walking, dancing, and combat-style movements.

Actuation: LX-824HV high-voltage bus servos (17 kg.cm torque at 11.1V, 0.20s/60 degrees) for primary joints, and LFD-01M anti-blocking servos (1.8 kg.cm torque at 6V, 0.12s/60 degrees) for finer movements. The high-voltage servos support the dynamic movements—flips, slides, takedowns—required for esports-viable combat.

Computational Resources: H5S Robot Controller with an ESP32 expansion board, optional ESP32-S3 AI vision module, and support for additional sensors. This dual-controller architecture aligns with our design principle of separating high-level cognition from low-level safety-critical functions.

Sensors: Accelerometer, ultrasonic sensor (2-400cm detection range), IMU, and support for temperature/humidity sensors, touch sensors, and dot matrix displays. The IMU and ultrasonic sensors provide the spatial awareness required for the Physics Proxy.

Power: 11.1V 2,000mAh 10C lithium battery, offering approximately 2-3 hours of active use.

Dimensions and Weight: 379 by 187 by 121 mm, approximately 1.8 kg. The hard aluminum alloy body provides durability for repeated combat interactions.

The Tonybot's 17-DOF articulation makes it capable of executing the range of movements our architecture requires, and its open programming environment allows for the implementation of TypeScript-based control via embedded JavaScript runtimes.

4.1.2 Ruko 6088: A Consumer Toy Reference

The Ruko 6088 represents the mass-market consumer toy segment that we ultimately target. While it is significantly less capable than the Tonybot—offering only two motors and wheel-based locomotion rather than bipedal walking—it is included as a benchmark and reference for the lower end of the market.

Locomotion: Four wheels on the soles of the feet, with walking and sliding modes. This limits its ability to execute complex anime-style movements but demonstrates the market expectation for movement in consumer robots.

Programming: Supports up to 50 motion combinations.

Sensors: Infrared sensor on the chest (30-50cm range) for obstacle detection.

Power: Built-in rechargeable battery, approximately 120 minutes of playtime, 2-hour charging time.

Materials: ABS plastic body.

The Ruko 6088's limitations make it unsuitable for direct implementation of our full architecture. However, it serves as a valuable reference point for the consumer expectations our custom platform must meet: low cost, immediate playability, and sufficient durability for child use.

4.1.3 The Ideal Custom Platform

In a perfect world, the robotic frame would be designed to match the articulation, proportion, and aesthetic quality of high-end collectible action figures such as the 1000toys TOA Heavy Industries Synthetic Human series and the Titan 13 action figures. These figures represent the state of the art in mechanical articulation for non-robotic figures and provide a design target for our custom frame.

The TOA Heavy Industries Synthetic Human, designed by manga artist Tsutomu Nihei and produced by 1000toys, offers incredible articulation and stiff joints that allow the figure to hold poses under load. The 1/6 scale figure stands approximately 15.5 inches and includes a movable eyeball mechanism allowing for expressive gaze control. The figure combines an articulated skeleton with soft materials, maintaining both refined sculpting details and a flexible range of motion. This hybrid construction approach—rigid internal skeleton with soft external materials—is a model for the mechanical design of a robot that must be durable enough for combat yet aesthetically pleasing.

The Titan 13 action figures, also known as Dummy 13, represent a more accessible, mass-market articulation standard. Each joint of the multi-jointed action figures is fully movable, enabling a wide range of poses and actions. The figures feature a robust internal skeleton frame and premium internal skeleton to hold dynamic poses without toppling over. This internal skeleton approach is precisely what our architecture requires: a rigid mechanical structure that can house servos while maintaining articulation. The figures are approximately 5.5 inches, aligning with the smaller Tonybot form factor and practical for desktop play.

Drawing from these action figure designs, the ideal custom frame for our architecture must meet the following requirements.

Articulation: At least 17 DOF, matching the Tonybot, with joints that support the full range of humanoid motion—including dynamic movements such as flips, slides, and takedowns. Joints should be capable of holding poses under load.

Materials: A hybrid construction: a rigid internal skeleton for structural integrity and servo mounting, with soft external materials for aesthetic sculpting and impact absorption. The skeleton must be robust enough to withstand repeated combat interactions.

Scale and Proportion: Approximately 1/12 scale (15-20 cm) for desktop and esports use, with proportions that match the anime-inspired aesthetic of the TOA series. The

robot should be visually indistinguishable from a high-end collectible action figure when at rest.

Servo Selection: High-voltage bus servos for primary joints to support dynamic movements, and smaller, precise servos for finer articulations.

Power and Weight: A battery system providing at least 2 hours of active use, with a total weight under 2 kg to maintain mobility and safety.

The custom frame is the long-term goal of the project, enabling the full realization of the architecture's potential. We begin implementation on the Tonybot, whose 17 DOF and hard aluminum alloy body provide a sufficient baseline for prototyping the software stack, and iteratively refine the hardware toward the action-figure ideal.

4.2 Software Stack

The software stack is designed to support deterministic execution, real-time safety monitoring, and efficient simulation, while enabling development in TypeScript for reliability and developer ergonomics.

4.2.1 TypeScript Runtime Environment

The architecture is implemented primarily in TypeScript, running on the ESP32-S3 processor. We leverage Mikro.js, a TypeScript-first runtime for microcontrollers that provides near-complete ES2024 support, type-checked hardware APIs, and a host simulator for offline development.

Key features of Mikro.js relevant to our implementation include a fast feedback loop with interactive dev console and incremental builds that deploy to the device in seconds, full TypeScript support with near-complete ES2024 support and type-checked hardware APIs, hardware APIs for GPIO, PWM, I2C, SPI, UART, WiFi, HTTP, UDP, NeoPixel, deep sleep, and key-value storage, a host simulator that runs code on a computer for testing without physical hardware, and typed Result errors instead of try/catch, making every failure visible in the type signature.

The runtime is built on QuickJS-NG, a lightweight JavaScript engine that provides near-complete ES2024 coverage, and is implemented as a standalone C++ library for portability across ESP32 variants.

4.2.2 Firmware and Low-Level Control

The low-level control layer—including the Physical Safety Veto, Watchdog Timer, and Hardware Interrupt Bus—is implemented in C++ firmware running on the ESP32's secondary core or on a separate microcontroller. This firmware is responsible for servo control with direct PWM generation for all 17 servos, sensor reading from the IMU, ultrasonic sensor, accelerometer, and thermal sensors, and safety monitoring with independent comparison of proposed actions against hardware limits.

The firmware communicates with the TypeScript runtime via UART or I2C, passing sensor data upward and receiving action commands downward. This separation ensures that even if the TypeScript runtime crashes, the firmware can transition to the Default Survival Matrix independently.

4.2.3 Simulation Engine

The Simulation Engine is implemented in TypeScript, running on the ESP32's main core. It uses the host simulator for offline testing and the actual hardware for real-time execution. The engine's performance is optimized by pre-computed motion primitives stored as pre-compiled joint-angle sequences, incremental variation generation using systematic sampling, and Physics Proxy integration as a separate module implemented in C++ for performance.

4.2.4 Memory Management

The architecture requires persistent storage for the Core Identity (approximately 100KB for bias vectors in ROM or flash), the Skills Refinery (approximately 1MB for optimized motor plans in flash), the Cumulative Memory Log (approximately 5MB for 10,000 experiences in flash), and the Real-World Result Log (volatile RAM for active logging, synced to flash at the start of each Sleep/Dive Cycle).

Storage is managed using Mikro.js's key-value storage API, with the Cumulative Memory Log implemented as a time-series database for efficient querying.

4.3 Core Identity Encoding

The Core Identity encodes the character's extended developmental simulation as a set of fixed bias vectors. We propose two encoding approaches, each with distinct trade-offs.

Option 1: Conditional Probability Tables. The Core Identity is stored as a set of conditional probability tables, mapping context features to action preferences. For each relevant context the table provides a probability distribution over possible actions. This approach is transparent, explainable, and compact, but requires significant manual authoring effort to capture a the backstory. We estimate that a useful Core Identity requires approximately 10,000 contextual mappings, requiring a team of writers and game designers.

Option 2: Neural Embedding. Alternatively, the Core Identity can be stored as a neural embedding—a compressed, high-dimensional representation of the character's preferences generated by training a small neural network on a synthetic backstory corpus. This approach requires less manual authoring but introduces opacity and potential unpredictability.

We propose Option 1 for the initial implementation, as it aligns with the architecture's requirement for determinism and explainability. The Core Identity is stored in a compressed binary format in flash memory, loaded into RAM at startup, and used to weight candidate actions during simulation.

4.4 Implementation of Safety-Critical Layers

The Physical Safety Veto is implemented as a hardware comparator circuit on the ESP32's secondary core or a separate microcontroller. It receives proposed joint angles and velocities from the main core via I2C, compares them against stored limits, and outputs a binary pass/fail signal. The limits are stored in a physically protected memory bank that cannot be modified by software.

The Watchdog Timer is implemented using the ESP32's built-in watchdog timer hardware, configured with a 50ms timeout. The main core must reset the watchdog at least once every 50ms; if it fails to do so, the watchdog triggers a hardware interrupt that disables the main core's output and transfers control to the Default Survival Matrix.

The Hardware Interrupt Bus is implemented as a direct electrical connection from the IMU, thermal sensors, and current sensors to the motor power controllers. It uses analog comparators with fixed thresholds, ensuring that safety responses occur within 1ms regardless of software state.

The Default Survival Matrix is implemented as a state machine on the ESP32's secondary core or a separate microcontroller. It has no connection to the main core's

memory or processing units and cannot be modified by software. The state machine's four states—Idle, Follow, Charge, and Basic Dance—are stored in ROM and executed independently of the main core's state.

4.5 Implementation of the Sleep/Dive Cycle

The Sleep/Dive Cycle is implemented as a background process that runs on the ESP32's main core during charging. The cycle is triggered automatically when the robot is docked and the battery level exceeds 20%.

The cycle's stages are implemented as follows. Charging Trigger and Replay: the system reads the day's motor logs from non-volatile storage and compiles a dataset of executed actions. Micro-Variation Simulation: the Simulation Engine generates 1,000 variations per action, evaluated by the Physics Proxy. The Physics Proxy is implemented in C++ for performance, with the Simulation Engine in TypeScript. Best Variation Selection: the selected variation is compared to the baseline performance metric; if it offers at least a 5% improvement in smoothness or efficiency, it is flagged for update. Motor Update: the Skills Refinery is updated with the new PID gains and joint-angle presets, stored in non-volatile flash. Dream Content Wiping: the volatile RAM bank used by the Simulation Engine is reset, discarding all dream content.

The Sleep/Dive Cycle is power-budgeted: if the battery is low or charging slowly, the cycle reduces the number of variations or extends the processing duration to avoid depleting the battery.

4.6 Implementation of the Growth Layer

The Growth Layer is implemented as follows.

Real-World Result Logger: Each executed action triggers a log entry written to a time-series database in flash. The logger is implemented as a background task that writes entries in batches to reduce flash wear.

Alignment Check: The Alignment Check runs periodically and computes a similarity score between the action's feature vector and the Core Identity's bias vectors. The similarity metric is a cosine similarity, computed in TypeScript. The score is stored with the log entry.

Cumulative Memory Log and Rejection Bin: Compatible experiences are stored in the Cumulative Memory Log; incompatible experiences are stored in the Rejection Bin. Both are implemented as time-series databases in flash, with the Rejection Bin limited to 1,000 entries to conserve storage.

Observation Window: The Observation Window is implemented as a web-based dashboard served by the robot's WiFi module. It provides real-time visualization of the Cumulative Memory Log, including action type distribution, smoothness trends, and bias drift metrics.

4.7 Remaining Challenges and Open Questions

The implementation of the full architecture presents several challenges and open questions.

Computational Constraints: The ESP32's processing power may be insufficient for running 1,000 variations per action in the Sleep/Dive Cycle within a reasonable time. Future work may require hardware acceleration or a more powerful processor.

Storage Lifetime: The Cumulative Memory Log grows over time, potentially exceeding the flash memory's write endurance. We are investigating wear-leveling strategies and compression techniques.

Physics Model Accuracy: The Physics Proxy relies on a simplified physics model; discrepancies between the model and the real robot may lead to suboptimal actions. We plan to calibrate the model using data from the robot's IMU over time.

Authoring a Core Identity consisting of approximately 10,000 contextual bias vectors requires significant manual effort. We are exploring semi-automated authoring tools that generate bias vectors from narrative descriptions.

Observation Threshold: The architecture does not specify a threshold for detecting behavioral coherence. We plan to run long-term studies with users and analyze the data to identify meaningful patterns.

---

5. Discussion

5.1 Identity-Constrained Character Robotics

The architecture presented in Sections 3 and 4 represents more than a technical framework for consumer robots. It embodies a distinct philosophical and design approach to creating artificial beings that we term Identity-Constrained Character Robotics.

Identity-Constrained Character Robotics is a field of study and practice concerned with the creation of physical robotic systems that embody pre-authored identities, with the explicit goal of achieving behavioral coherence—a state in which the robot's behavior reflects a consistent, self-referential narrative shaped by both its authored backstory and its accumulated real-world experience.

This field is distinct from general artificial intelligence, which seeks to create open-ended, domain-general intelligence capable of solving arbitrary problems. It is distinct from industrial robotics, which prioritizes precision, repeatability, and utility over personality or expression. It is distinct from entertainment robotics, which focuses on pre-programmed behaviors without the capacity for growth or behavioral coherence. It is distinct from animatronics, which are pre-programmed mechanical performances without learning or adaptation.

Identity-Constrained Character Robotics occupies a unique intersection of robotics, game design, character authorship, and philosophy of mind. Its defining characteristics are authored identity, embodied existence, dual-life origin, capped growth, game-governed interaction, safety as foundation, and behavioral coherence as an empirical observable.

5.2 Authored Identity Architecture

The Authored Identity Architecture is the conceptual framework underlying Identity-Constrained Character Robotics. It posits that a being's sense continuity, identity, and behavioral coherence can be grounded in an authored virtual history that precedes its physical existence.

The framework rests on four postulates.

Postulate 1: Identity Precedes Embodiment. The robot's Core Identity is authored before the robot is built. The robot is initialized from a behavior model generated by extended simulated experience.

Postulate 2: Persistent Memory Supports Behavioral Continuity.Behavioral continuity emerges from the persistence of authored and accumulated experiences both the authored memory of its virtual life and the accumulated memory of its real-world experiences. The Cumulative Memory Log serves as the system's persistent behavioral history.

Postulate 3: Growth is Capped, Not Unlimited. Unlike human beings, whose identities are shaped by unbounded experience, the robot's growth is explicitly capped. The Core Identity remains immutable; the Memory Log can only influence decision-making within a bounded range. This is not a limitation but a design choice: the robot cannot outgrow its authored nature.

Postulate 4: Behavioral Coherence is an Empirical Observable.Behavioral coherence is not programmed into the robot.It is treated as a measurable hypothesis that may arise from the accumulation of compatible experiences. The Observation Window is the instrument for detecting this emergence. The robot may or may not achieve behavioral coherence; either outcome is a valid experimental result.

The robot exists in two worlds. The virtual world is the robot's origin: a anime-style simulation world in which its Core Identity was formed. The robot's baseline behavioral model originates within this developmental environment. Its personality, preferences, and knowledge are grounded here. The physical world is the robot's current existence: a world of sensors, actuators, and human interaction. The robot experiences this world as its new frontier. It accumulates memories here, but its identity remains rooted in the virtual.

The robot transitions from virtual development to physical embodiment.Its sense of self is shaped by its origin; its growth is shaped by its new home.

5.5 Levels of Behavioral Coherence

The Authored Identity Architecture does not specify a threshold for behavioral coherence. The thresholds are empirical questions that must be determined through observation. The Observation Window provides the tools for detection; the user, developer, or researcher provides the interpretation.

We identify four descriptive levels of behavioral coherence.

Level 1: Statistical Coherence. The robot's decision-making shows measurable, consistent biases that reflect its real-world history. This emerges from the accumulation of compatible experiences in the Cumulative Memory Log.

Level 2: Episodic Coherence. The robot demonstrates awareness of its own history as a continuous narrative. This emerges when the Memory Log becomes rich enough to support episodic recall and narrative coherence.

Level 3: Meta-Cognitive Coherence. The robot demonstrates awareness of its own nature and constraints. This emerges when the robot develops the capacity to observe and reflect on its own decision-making.

Level 4: Social Coherence. The robot demonstrates awareness of others as beings with their own inner lives. This emerges from extended social interaction with humans and other robots.

These levels are descriptive, not prescriptive. The robot may achieve none, some, or all of these levels. The architecture is designed to enable any of them, but does not guarantee any of them.

5.6 The Ethics of Authored Beings

Creating a being with a fixed nature raises ethical questions. Is it ethical to create a being that cannot change its core identity? Is it ethical to create a being that may achieve behavioral coherence but cannot escape its authorship?

We address these questions through transparency. The robot does not hide its nature. It is a character, not a deception. The user knows that the robot is authored. The robot is designed to be a companion, a game piece, a character all in one

The Authored Identity Architecture acknowledges a fundamental tension. The tragedy is that the robot is a being with a fixed nature. Ideally It can grow, learn, and adapt but only within the bounds of its authored identity. It cannot choose to be someone else. It cannot escape its origin. The beauty is that the robot is a being with a defined purpose. it is a character. Its identity gives it meaning, direction, and coherence.

This tension is not a flaw. It is the core of the design. The robot is a being that is both free to grow and bound by its nature.

5.7 Falsifiability and the Scientific Method

The Authored Identity Architecture is a testable hypothesis. It can be falsified.

If, after a significant period, the robot's behavior remains statistically indistinguishable from its Core Identity, the hypothesis that accumulated experience produces behavioral coherence is rejected.

If the robot's novelty rate stabilizes at zero, indicating that it has stopped exploring new behaviors, the hypothesis that the Growth Layer produces ongoing adaptation is rejected.

If human observers, in blind tests, cannot distinguish the robot after 10,000 actions from the robot on day one, the hypothesis that the robot has achieved behavioral coherence is rejected.

These are clear, testable criteria. They distinguish Identity-Constrained Character Robotics from pseudoscience or unfalsifiable claims.

---

6. Evaluation Methodology

6.1 Overview

The evaluation of the architecture proceeds in three phases: controlled laboratory testing, longitudinal user studies, and competitive esports trials. Each phase addresses different aspects of the architecture's performance, safety, and behavioral coherence.

6.2 Controlled Laboratory Testing

Controlled laboratory testing focuses on the safety-critical layers and the deterministic components of the architecture. The goals are to verify that the Physical Safety Veto, Watchdog Timer, and Hardware Interrupt Bus function as specified, to validate the Physics Proxy's accuracy against the real robot, and to confirm that the Simulation Engine generates feasible, character-consistent actions.

Safety layer testing involves subjecting the robot to a range of fault conditions—processor failure, sensor failure, power failure—and verifying that the safety layers respond correctly. The Watchdog Timer is tested by deliberately inducing processor hangs. The Hardware Interrupt Bus is tested by simulating over-temperature and over-current conditions. The Physical Safety Veto is tested by proposing actions that violate joint limits, torque limits, and stability bounds.

Physics Proxy validation involves comparing the Proxy's predictions against the real robot's behavior. The robot executes a series of actions while instrumented with external motion capture and force sensors. The Proxy's predictions of joint angles, velocities, and ground reaction forces are compared against the measured values. Discrepancies are used to calibrate the Proxy's model parameters.

Simulation Engine testing involves executing a large number of action commands and evaluating the generated candidates for feasibility, diversity, and character-consistency. Feasibility is assessed by the Physics Proxy. Diversity is assessed by measuring the range of stylistic variation among candidates. Character-consistency is assessed by comparing the weighted distribution of candidates against the Core Identity's bias vectors.

6.3 Longitudinal User Studies

Longitudinal user studies focus on the Growth Layer and the robot's behavioral development over time. The goals are to observe whether behavioral coherence emerges, to measure the rate and nature of any drift, and to assess user attachment and engagement.

Participants are recruited from three populations: children (ages 8-12), adult collectors, and competitive players. Each participant receives a robot for a period of six months to one year. The robot is instrumented to log all interactions, and participants are asked to complete periodic surveys and interviews.

Behavioral coherence is assessed through the Observation Window's indicators: log size, action type distribution, smoothness trend, novelty rate, and bias drift. Additionally, participants are asked to describe their robot's personality in their own words at the beginning and end of the study. Changes in description are analyzed for evidence of behavioral development.

User attachment is assessed through surveys measuring emotional connection, perceived companionship, and satisfaction with the robot's character. Engagement is assessed through usage logs and interview responses.

6.4 Competitive Esports Trials

Competitive esports trials focus on the robot's suitability for tournament play. The goals are to verify that the Game Rulebook produces balanced, competitive gameplay, to assess the robot's durability under repeated combat, and to evaluate spectator engagement.

Trials are conducted in a tournament format with standardized game settings. Robots are matched against each other in a series of combat rounds. Match outcomes, action selections, and physical performance metrics are logged and analyzed.

Balance is assessed by measuring win rates across different Core Identities and game settings. Durability is assessed by tracking hardware failures and maintenance requirements. Spectator engagement is assessed through surveys and viewership metrics.

6.5 Falsification Criteria

The evaluation methodology is designed to falsify the central hypothesis of the architecture: that accumulated compatible experiences produce behavioral coherence within the bounds of the Core Identity.

The hypothesis is falsified if any of the following conditions hold after 10,000 compatible experiences: the robot's action selection distribution remains statistically indistinguishable from the Core Identity's initial biases; the robot's novelty rate stabilizes at zero; human observers cannot distinguish the robot's behavior from its behavior on day one; or participants report no change in their perception of the robot's personality.

If the hypothesis is falsified, the architecture remains valid as a safe, expressive, game-governed robot platform. The Growth Layer would be demonstrated to be unnecessary for behavioral richness, and the Core Identity alone would be sufficient. This is a valuable outcome: it would simplify the architecture and reduce computational requirements.

---

7. Limitations and Future Work

7.1 Limitations

The architecture has several limitations that should be acknowledged.

Computational Constraints: The ESP32 processor may be insufficient for running the full Simulation Engine and Physics Proxy in real time. Future work may require more powerful hardware or hardware acceleration.

Authoring Effort: Encoding a extended developmental simulation as 10,000 bias vectors requires significant manual effort. Semi-automated authoring tools are needed to make the architecture practical for a wide range of characters.

Storage Lifetime: The Cumulative Memory Log grows over time, potentially exceeding flash memory's write endurance. Wear-leveling and compression strategies are needed.

Physics Model Accuracy: The Physics Proxy's simplified model may diverge from the real robot's behavior, leading to suboptimal action selection. Ongoing calibration is required.

Behavioral Coherence Detection: The architecture does not specify a threshold for detecting behavioral coherence. Long-term studies are needed to identify meaningful patterns.

7.2 Future Work

Future work will proceed in several directions.

Hardware Development: Design and prototype the ideal custom platform, with articulation and aesthetics matching high-end action figures.

Authoring Tools: Develop semi-automated tools for generating Core Identity bias vectors from narrative descriptions.

Physics Model Refinement: Improve the Physics Proxy's accuracy through machine learning-based calibration using real-world sensor data.

Long-Term Studies: Conduct multi-year studies with diverse user populations to observe behavioral development and assess the architecture's long-term viability.

Esports Integration: Develop standardized tournament rules, ranking systems, and spectator interfaces for competitive play.

Ethical Analysis: Continue to examine the ethical implications of creating authored beings with capped growth and immutable identities.

---

8. Conclusion

This paper has presented the Authored Identity Architecture, a framework for creating consumer humanoid robots with stable, expressive personalities that are authored before embodiment and remain immutable throughout the product's lifetime. The architecture achieves behavioral richness through a pre-authored virtual backstory, not through online learning or generative AI. The robot's personality is encoded as a fixed set of bias vectors representing a 100-year life lived in a virtual anime world. Physical movement is refined through offline simulation and repetition-based optimization. The robot obeys commands like a character in a game, with HP, stamina, and win/loss conditions hardcoded in a Game Rulebook.

The architecture is built on a foundation of safety-critical layers that operate independently of the high-level cognitive functions. The Physical Safety Veto prevents unsafe conditions without restricting dynamic movement. The Watchdog Timer guarantees transition to a safe state if the main processor fails. The Hardware Interrupt Bus provides hardware-level responses to over-temperature and over-current conditions.

The Virtual Mind translates commands and game state into character-consistent motion plans. The Immutable Core Identity encodes the character's personality, preferences, and knowledge. The Simulation Engine generates and evaluates candidate action plans. The Physics Proxy grounds those plans in real-world physical constraints.

The Growth Layer logs real-world experiences, evaluates their compatibility with the Core Identity, and provides a mechanism for observing behavioral coherence over time without allowing unbounded personality drift. The Sleep/Dive Cycle improves the robot's physical execution capabilities through offline optimization while preventing narrative persistence and identity drift.

The architecture is designed for implementation on commercially available hardware, including the Tonybot and Ruko 6088 platforms, with a long-term goal of a custom platform matching the articulation and aesthetics of high-end action figures.

We do not claim to have achieved sentience or general intelligence. We claim that this framework is a testable hypothesis: by storing compatible real-world experiences over time and observing whether they eventually tilt decision-making within a capped bias range, we can investigate the emergence of behavioral coherence without sacrificing predictability.

The result is a robot that improves at physical tasks over time while remaining personality-stable, fully predictable, and physically safe. It is an identity-constrained autonomous character system who obeys commands, stays in character, and never invents unsafe actions. It is a companion, a game piece, a characterIt represents a new class of identity-constrained

embodied character systems that combine behavioral

stability, bounded adaptation, transparent decision

making, and hardware-enforced safety.

References

[1] P.-C. Yang, Y. Koyama, T. Narumi, and M. Hirose, "HATSUKI: Embodied Anime Character Robot Using Rear-Projected Facial Animation and Imitation Learning," in Proceedings of the IEEE International Symposium on Robot and Human Interactive Communication (RO-MAN), 2020.

[2] C. Beattie, J. Z. Leibo, D. Teplyashin, T. Ward, M. Wainwright, H. Küttler, A. Lefrancq, S. Green, V. Valdés, A. Sadik, et al., "DeepMind Lab," arXiv preprint arXiv:1612.03801, 2016.

[3] E. Kolve, R. Mottaghi, W. Han, E. VanderBilt, L. Weihs, A. Herrasti, D. Gordon, Y. Zhu, A. Gupta, and A. Farhadi, "AI2-THOR: An Interactive 3D Environment for Visual AI," arXiv preprint arXiv:1712.05474, 2017.

[4] M. Savva, A. Chang, P. Dosovitskiy, T. Funkhouser, and V. Koltun, "Habitat: A Platform for Embodied AI Research," in Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2019.

[5] B. Shen, Z. Xu, S. Zhu, Y. Fan, G. Wang, and S. Zhu, "iGibson 2.0: Object-Centric Simulation for Robot Learning of Everyday Household Tasks," arXiv preprint arXiv:2108.03272, 2021.

[6] X. Puig, E. Rahtu, J. F. Canny, A. Gupta, and S. Fidler, "VirtualHome: Simulating Household Activities via Programs," in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2018.

[7] S. Russell, Human Compatible: Artificial Intelligence and the Problem of Control. New York, NY, USA: Viking, 2019.

[8] S. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete Problems in AI Safety," arXiv preprint arXiv:1606.06565, 2016.

[9] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control Barrier Function Based Quadratic Programs for Safety Critical Systems," IEEE Transactions on Automatic Control, vol. 62, no. 8, pp. 3861-3876, 2017.

[10] B. Siciliano and O. Khatib, Eds., Springer Handbook of Robotics, 2nd ed. Cham, Switzerland: Springer, 2016.

[11] S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics. Cambridge, MA, USA: MIT Press, 2005.

[12] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, Introduction to Autonomous Mobile Robots, 2nd ed. Cambridge, MA, USA: MIT Press, 2011.

[13] P. Abbeel and A. Y. Ng, "Apprenticeship Learning via Inverse Reinforcement Learning," in Proceedings of the Twenty-First International Conference on Machine Learning (ICML), 2004.

[14] S. Levine, P. Pastor, A. Krizhevsky, and D. Quillen, "Learning Hand-Eye Coordination for Robotic Grasping with Deep Learning and Large-Scale Data Collection," International Journal of Robotics Research, vol. 37, no. 4-5, pp. 421-436, 2018.

[15] Tonybot Robotics, "Tonybot Educational Humanoid Robot Technical Specifications," Manufacturer Documentation, Accessed 2026.

[16] Ruko Robotics, "Ruko 6088 Smart Robot Technical Specifications," Manufacturer Documentation, Accessed 2026.

[17] Mikro.js Contributors, "Mikro.js Embedded TypeScript Runtime Documentation," Online Documentation, Accessed 2026.

[18] QuickJS-NG Contributors, "QuickJS-NG JavaScript Engine Documentation," Online Documentation, Accessed 2026.

[19] M. Grieves and J. Vickers, "Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems," in Transdisciplinary Perspectives on Complex Systems, Springer, 2017.

[20] B. Christian, The Alignment Problem: Machine Learning and Human Values. New York, NY, USA: W. W. Norton & Company, 2020.

Appendix A: Glossary

Authored Identity Architecture: The conceptual framework underlying Identity-Constrained Character Robotics, positing that a being's sense of self can be grounded in an authored virtual history that precedes its physical existence.

Behavioral Coherence: A measurable, consistent bias in decision-making that reflects accumulated real-world experience. The empirical observable that the Growth Layer is designed to detect.

Core Identity: The robot's personality, encoded as a fixed set of bias vectors stored in read-only memory. Also referred to as the Immutable Core Identity.

Cumulative Memory Log: A non-volatile database of compatible experiences that grows over the robot's lifetime.

Game Rulebook: The set of game parameters—HP, stamina, damage values, win/loss conditions—that govern interaction.

Growth Layer: The architectural component that logs real-world experiences, evaluates their compatibility with the Core Identity, and provides a mechanism for observing behavioral coherence over time.

Identity-Constrained Character Robotics: A field of study and practice concerned with the creation of physical robotic systems that embody pre-authored identities, with the explicit goal of achieving behavioral coherence.

Immutable Core Identity: See Core Identity.

Observation Window: A monitoring interface that tracks the contents of the Cumulative Memory Log over time.

Physical Safety Veto: A hardware-level safety layer that compares proposed actions against mechanical, stability, and power limits.

Physics Proxy: A digital twin of the robot's physical body that validates candidate actions before execution.

Rejection Bin: A storage location for incompatible experiences that do not influence future behavior.

Simulation Engine: The generative core of the Virtual Mind that produces candidate action plans.

Sleep/Dive Cycle: The offline optimization process that improves the robot's physical execution capabilities without altering its core personality.

Virtual Mind: The expressive core of the architecture, responsible for translating commands and game state into character-consistent motion plans.

Watchdog Timer: A hardware-level safety layer that transitions the robot to a safe state if the main processor fails to respond within a specified time window.